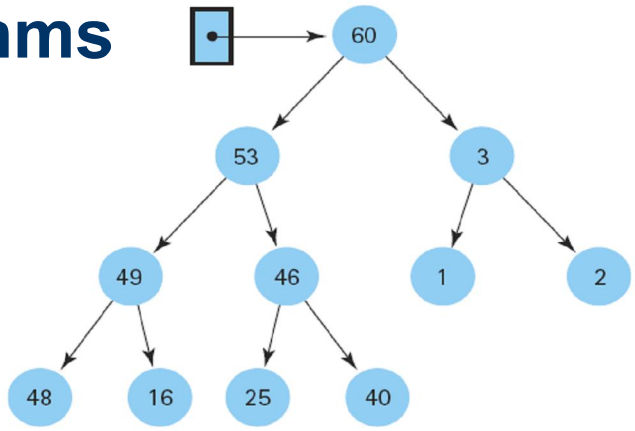
Data Structures & Algorithms



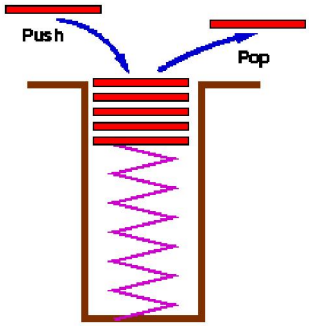
Stacks

•Linear data structure which can be accessed only at one time of it's ends for storing and retrieving data

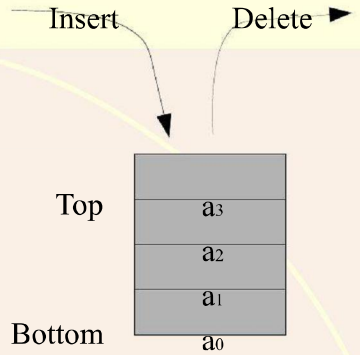
• Widely used data structures

• Easy to implement

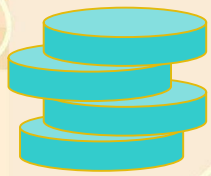
• Easy to use



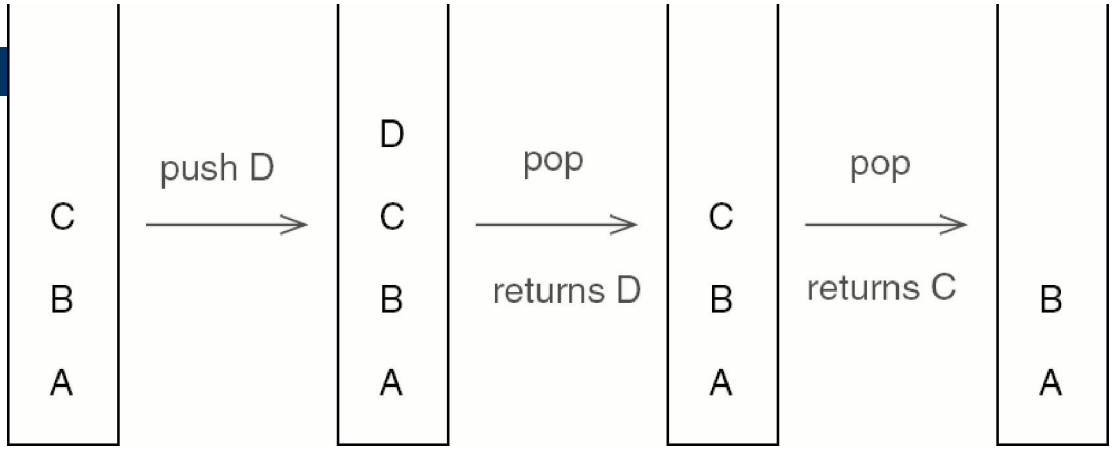
Stacks



- $S=(a_0,\ldots,a_{n-1})$
  - a_0 is the bottom of the stack
  - a_{n-1} is the top of the stack
  - n is the number of element
- Insertions and deletions are made at the top
- *Last In First Out (LIFO) list*
  - Example: stack of plates



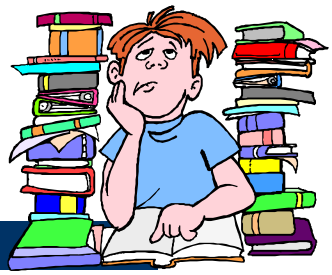
The Stack



Describe the output of the following series of stack operations

Push(8)
Push(3)
Pop()
Push(2)
Push(5)
Pop()
Pop()
Push(9)
Push(1)

The Stack Operation



- Insertions and deletions follow the last-in first-out (LIFO) scheme
- **Main stack operations:**
  - push(Object o): inserts element o
  - pop(): removes and returns the last inserted element
- **Auxiliary stack operations:**
  - top(): returns the last inserted element without removing it
  - size(): returns the number of elements stored
  - isEmpty(): a Boolean value indicating whether no elements are stored
  - isFull() (a Boolean value indicating whether a stack is full or not)

Stack Applications

1. Reversing Data: We can use stacks to reverse data.
   (example: files, strings)
   Very useful for finding palindromes.

Consider the following pseudocode:

```
1)  read (data)
2)  loop (data not EOF and stack not full)
        1) push (data)
        2) read (data)
3)  Loop (while stack notEmpty)
        1) pop (data)
        2) print (data)
```

Stack Applications (cont)

2. history of visited pages through a Web browser
3. undo sequence in a word processor
4. sequence of method/function calls
5. Convert form infix to postfix or to prefix
6. Evaluate a postfix and prefix expression

arithmetic expressions :

- Postponement: Evaluating arithmetic expressions.

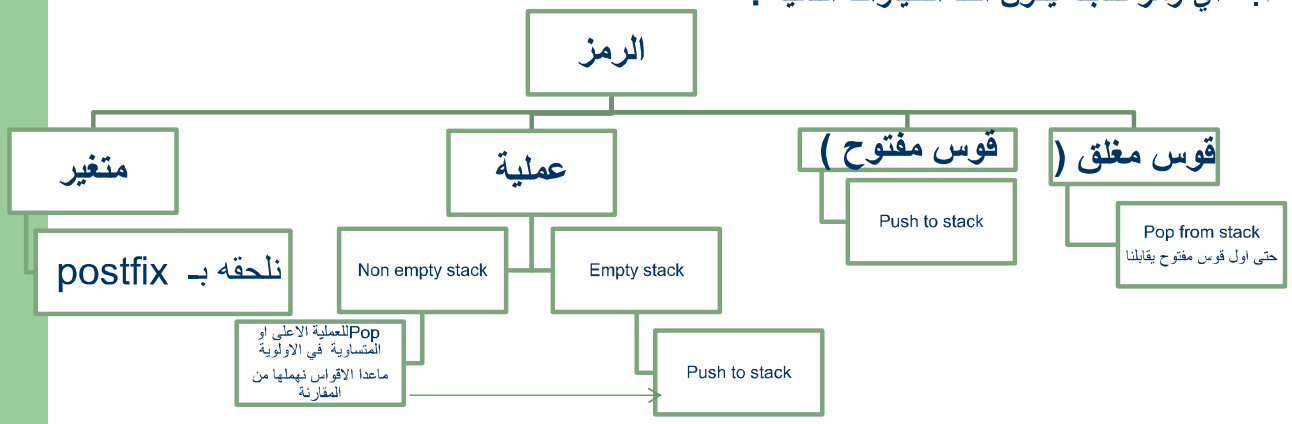
- Prefix: + a b
- Infix: a + b (what we use in grammar school)
- Postfix: a b +

- In high level languages, infix notation cannot be used to evaluate expressions. We must analyze the expression to determine the order in which we evaluate it. A common technique is to convert a infix notation into postfix notation, then evaluating it.

So Postfix expressions using in compilers

Convert to postfix :

١. نقوم بمسح جميع الرموز من اليسار لليمين
٢. اي رمز نقابله يكون احد الخيارات التالية :



٣. اذا تبقى رموز في stack نقوم بعمل pop ونلحقها بـ postfix حتى يصبح empty stack

How to convert the following infix to postfix ?

A * (B + C) – D / E

stack	postfix								
EMPTY									
EMPTY	A	B	C	+	*	D	E	/	-

How to evaluate the following expression ?

5 9 8 + 4 6 * * 7 + *

Stack in step 1	Stack in step 2	Stack in step 3	Stack in step 4	Stack in step 5	Stack in step 6	Stack in step 7	Stack in step 9	Stack in step 10	Stack in step 11	Stack in step 12

Exercise :

Change the following expression from infix to postfix :

1. (A * B) + (C - D / E)

ANS : A B * C D E / - +

2. 5 * (((9 + 8) * (4 * 6)) + 7)

ANS: 5 9 8 + 4 6 * * 7 + * Eval: 2075

3. A * (B + (C + D) * (E + F) / G) * H

ANS: A B C D + E F + * G / + * H *

4. 4 ^ 2 * 3 – 3 + 8 / 4 / (1 + 1)

ANS: 4 2 ^ 3 * 3 – 8 4 / 1 1 + / + Eval: 46

Stack implementation in c :

- Stacks structures are usually implemented using arrays or linked lists.

Push function :

```
#include<iostream>
using namespace std;
const int MAX=5;
void push(int stack[],int *top,int value)
{
    if (*top < MAX-1)
    {
        *top=*top+1;
        stack[*top]=value;
        return;
    }
    else
        cout<<"stack is full\n";
}
```

Pop function:

```
int pop(int stack[],int *top)
{
    int value;
    if (*top > -1)
    {
        value=stack[*top];
        *top=*top-1;
        return value;
    }

    else
        return -1;
}
```

Main function :

```
void main(){
    int stack[MAX],top=-1,value,choice;
    char answer;

    cout<<"1. to push element\n"  <<"2. to pop element\n";
    cout<<"want to try (y/n)\n";    cin>>answer;

    while(answer =='Y'||answer =='y')
    {
        cout<<"Enter your choise:\n"; cin>>choice;

        switch(choice)
        {
    case 1:cout<<"Enter the value to push:\n";cin>>value;
        push(stack,&top,value);         break;
    case 2:value=pop(stack,&top);
        if(value!=-1)cout<<"the value poped is "<<value<<endl;
        else cout<<"stack is empty\n";  break;
    default:cout<<"invalid choise\n";
        }
    cout<<"more ?(y/n)\n";  cin>>answer;
    }                                       }
```

END